

Week 6

Reproducible Environments

Yongqiang Tian

Department of Software Systems and Cybersecurity
Faculty of IT, Monash University

Acknowledgement

I wish to acknowledge the people of the Kulin Nations, on whose land we are gathered today. I pay my respects to their Elders, past and present.

Same code, different result

Both students run the same file

```
# app.py
import requests
print("FIT2109 lab")
```

Same code, different result

Both students run the same file

```
# app.py
import requests
print("FIT2109 lab")
```

Student A

```
$ python3 app.py
FIT2109 lab
```

Same code, different result

Both students run the same file

```
# app.py
import requests
print("FIT2109 lab")
```

Student A

```
$ python3 app.py
FIT2109 lab
```

Student B

```
$ python3 app.py
ModuleNotFoundError:
  No module named 'requests'
```

Same code, different result

Both students run the same file

```
# app.py
import requests
print("FIT2109 lab")
```

Student A

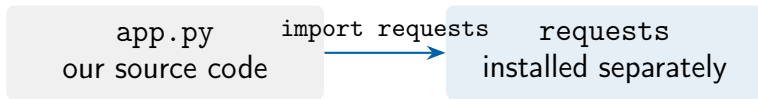
```
$ python3 app.py
FIT2109 lab
```

Student B

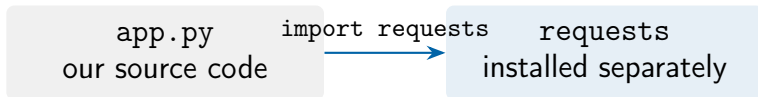
```
$ python3 app.py
ModuleNotFoundError:
  No module named 'requests'
```

Same source does not guarantee the same running conditions.
A rebuildable environment makes failures easier to explain.

What is a dependency?

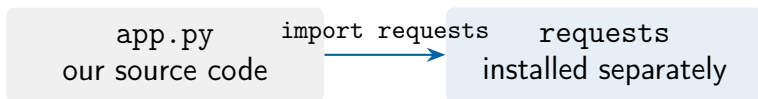


What is a dependency?



A **dependency** is software the project needs but did not implement itself.

What is a dependency?



A **dependency** is software the project needs but did not implement itself.

Student A's environment

`requests` installed

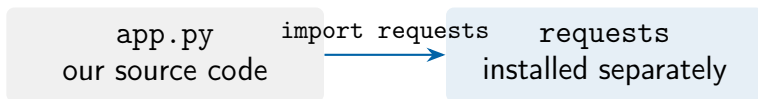
Python can complete the import.

Student B's environment

`requests` missing

Python raises `ModuleNotFoundError`.

What is a dependency?



A **dependency** is software the project needs but did not implement itself.

Student A's environment

`requests` installed

Python can complete the import.

Student B's environment

`requests` missing

Python raises `ModuleNotFoundError`.

The package is outside `app.py`, but it is still part of the program's running conditions.

Why versions differ

fictional package example

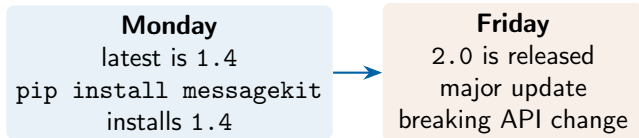
Monday

latest is 1.4

```
pip install messagekit  
installs 1.4
```

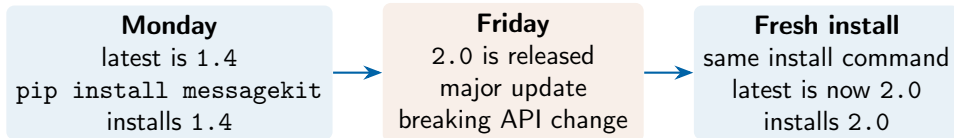
Why versions differ

fictional package example



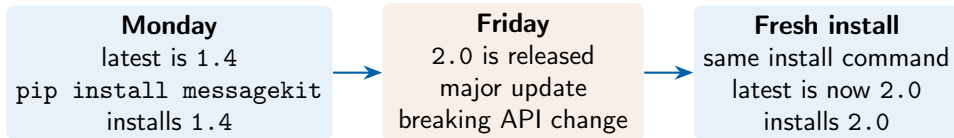
Why versions differ

fictional package example



Why versions differ

fictional package example



The application is unchanged

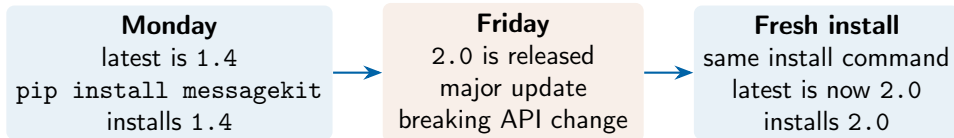
```
render("ready", colour="blue")
```

The selected API is different

```
1.4 -> ready  
2.0 -> TypeError:  
       unexpected argument 'colour'
```

Why versions differ

fictional package example



The application is unchanged

```
render("ready", colour="blue")
```

The selected API is different

```
1.4 -> ready  
2.0 -> TypeError:  
       unexpected argument 'colour'
```

The old environment did not update; a later install made a new choice.

Request one version

No version is specified

```
pip install messagekit
```

The latest available release is selected at install time.

Different install times → different versions.

Request one version

No version is specified

```
pip install messagekit
```

The latest available release is selected at install time.

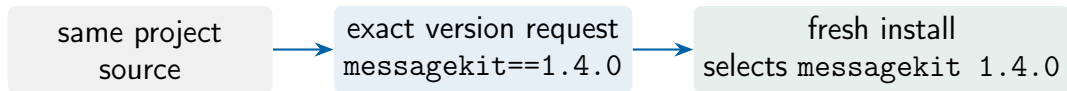
Different install times → different versions.

One exact version is specified

```
pip install messagekit==1.4.0
```

`==1.4.0` asks pip for exactly version 1.4.0.

Checkpoint: version choice



One version-choice problem is now explicit.

Next: how does a Python project record this request?

Using requirements.txt

Python projects often record package requirements in a file named `requirements.txt`.

Week 6 application

```
# requirements.txt
requests==2.31.0
```

Read this line as:

install requests, exactly version 2.31.0.

One line, three parts

requests	package name
==	exactly this version
2.31.0	requested version

Using requirements.txt

Python projects often record package requirements in a file named `requirements.txt`.

Week 6 application

```
# requirements.txt
requests==2.31.0
```

Read this line as:

install requests, exactly version 2.31.0.

One line, three parts

requests	package name
==	exactly this version
2.31.0	requested version

```
python3 -m pip install -r requirements.txt
-r requirements.txt: read requirements from this file.
```

Using requirements.txt

Python projects often record package requirements in a file named `requirements.txt`.

Week 6 application

```
# requirements.txt
requests==2.31.0
```

Read this line as:

install requests, exactly version 2.31.0.

One line, three parts

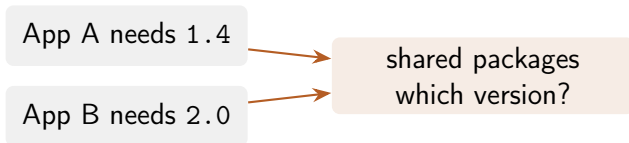
requests	package name
==	exactly this version
2.31.0	requested version

```
python3 -m pip install -r requirements.txt
-r requirements.txt: read requirements from this file.
```

pip3 is also common; `python3 -m pip` runs the pip belonging to that python3.

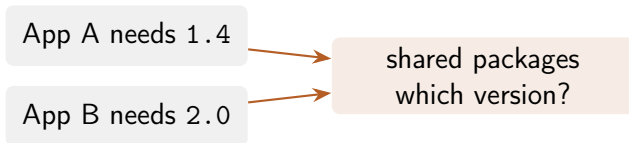
Why use a venv?

By default, both apps share one Python package location



Why use a venv?

By default, both apps share one Python package location



The shared version creates a conflict

shared 1.4: App A runs; App B fails.

shared 2.0: App A fails; App B runs.

One venv per project

Each project gets its own Python package location.

App A → `.venv-a` → `messagekit 1.4`

App B → `.venv-b` → `messagekit 2.0`

One venv per project

Each project gets its own Python package location.

```
App A → .venv-a → messagekit 1.4
```

```
App B → .venv-b → messagekit 2.0
```

A **virtual environment** is project-local Python package state, created from an existing Python installation.

Both versions can coexist; both apps can run.

Best practice: requirements + venv

Share the description; rebuild the environment locally.

1. Developer / team — record and commit the tested version

```
# requirements.txt  
requests==2.31.0
```

Best practice: requirements + venv

Share the description; rebuild the environment locally.

1. **Developer / team** — record and commit the tested version

```
# requirements.txt
requests==2.31.0
```

2. **Each user / teammate** — create and activate a local venv

```
python3 -m venv .venv
source .venv/bin/activate
```

Best practice: requirements + venv

Share the description; rebuild the environment locally.

1. **Developer / team** — record and commit the tested version

```
# requirements.txt  
requests==2.31.0
```

2. **Each user / teammate** — create and activate a local venv

```
python3 -m venv .venv  
source .venv/bin/activate
```

3. **The same user** — install dependencies into the active venv

```
python -m pip install -r requirements.txt
```

Best practice: requirements + venv

Share the description; rebuild the environment locally.

1. **Developer / team** — record and commit the tested version

```
# requirements.txt  
requests==2.31.0
```

2. **Each user / teammate** — create and activate a local venv

```
python3 -m venv .venv  
source .venv/bin/activate
```

3. **The same user** — install dependencies into the active venv

```
python -m pip install -r requirements.txt
```

Share requirements.txt; rebuild .venv for each local checkout.

Demo: create the venv

1. Read the recorded input

```
# requirements.txt  
requests==2.31.0
```

Demo: create the venv

1. Read the recorded input

```
# requirements.txt  
requests==2.31.0
```

2. Create, activate, and install

```
python3 -m venv .venv-demo  
source .venv-demo/bin/activate  
python -m pip install -r requirements.txt
```

Demo: create the venv

1. Read the recorded input

```
# requirements.txt  
requests==2.31.0
```

2. Create, activate, and install

```
python3 -m venv .venv-demo  
source .venv-demo/bin/activate  
python -m pip install -r requirements.txt
```

The file says **what**; the active venv determines **where**.

Demo: inspect the result

Run inside the active venv

```
python -c "import sys; print(sys.executable)"  
python -c "import requests; print(requests.__file__)"  
python -c "import sys; print(sys.base_prefix)"
```

Demo: inspect the result

Run inside the active venv

```
python -c "import sys; print(sys.executable)"
python -c "import requests; print(requests.__file__)"
python -c "import sys; print(sys.base_prefix)"
```

Evidence	What it shows
.../.venv-demo/bin/python	this shell uses the venv launcher
.../.venv-demo/.../requests/	the package is installed inside the venv
system Python path	the venv was created from an existing Python

Activity 1: is the venv active?

The project requires `requests==2.31.0`. Two terminals show their current Python and package state.

Terminal A

```
$ python -c "import sys; print(sys.executable)"  
.../week6/.venv/bin/python
```

```
$ python -m pip show requests  
Version: 2.31.0  
Location: .../week6/.venv/lib/.../site-packages
```

Terminal B

```
$ python -c "import sys; print(sys.executable)"  
/usr/local/bin/python3
```

```
$ python -m pip show requests  
Version: 2.32.4  
Location: /usr/local/lib/.../site-packages
```

6 minutes: Which terminal is ready to run the project? Circle two pieces of evidence. Then write the two commands Terminal B should run to use the project venv and install its requirements.

Python can call a system command

Here the system command is `curl`

`curl` is a separate command-line program for making network requests. It is not included with Python or a `venv`, and some machines do not have it installed.

```
import requests
import subprocess

print("requests", requests.__version__)
# ask the OS to run: curl --version
subprocess.run(["curl", "--version"], check=True)
```

When curl is not installed

```
$ python app.py  
requests 2.31.0  
FileNotFoundError: No such file or directory: 'curl'
```

Python package found

The venv supplied
requests 2.31.0.

System command missing

The OS could not find
the curl executable.

Two package managers

Python package

```
python3 -m pip install \  
requests==2.31.0
```

Installed for a selected Python environment.

Two package managers

Python package

```
python3 -m pip install \
    requests==2.31.0
```

Installed for a selected Python environment.

Ubuntu OS package

```
sudo apt update
sudo apt install curl
```

Installed into shared operating-system state.

Same command, different machines

```
setup guide says: apt install curl
```

Ubuntu release A

repository A
curl release A

Ubuntu release B

repository B
curl release B

macOS host

apt unavailable
different package manager

The written command is identical; each machine still determines what can be installed.

What the host still decides

The guide records one instruction: `apt install curl`

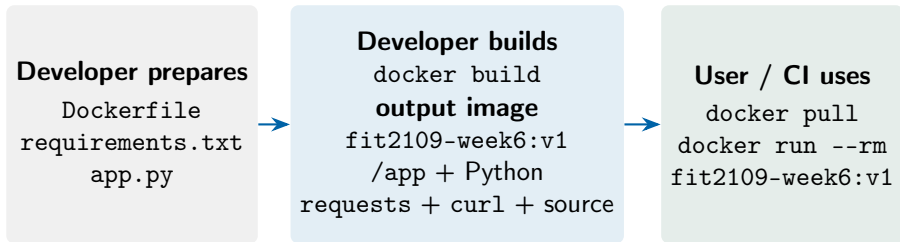
- **Operating system:** is `apt` available on this machine?
- **Repositories:** which `curl` release is available today?
- **Permissions and shared packages:** can the install proceed, and could it affect another application?

Same instruction does not guarantee the same installed environment.

Simple local setup: a guide may be enough.

Development and CI need one prepared environment: build and share an image.

From developer files to user run

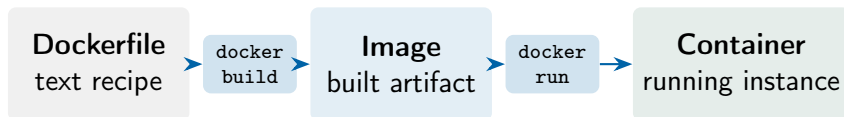


Handoff: a built image, not a list of manual setup steps.

The user pulls and runs it—no repeated apt, pip, or host-path setup.

In class we build locally; teams commonly publish versioned images through an image registry.

Three objects, two actions



An image can be reused. Each `docker run` creates a new container instance from it.

Dockerfile: the recipe

```
FROM python:3.12-slim
RUN apt-get update && apt-get install -y curl

WORKDIR /app
COPY requirements.txt .
RUN python -m pip install -r requirements.txt

COPY app.py .
CMD ["python", "app.py"]
```

FROM selects the base; apt and pip install dependencies; COPY adds source.

Stable internal path: WORKDIR /app gives the project the same path on every host.

Demo: build and verify

Before running: predict the two output lines.

```
docker build -t fit2109-week6 .  
docker run --rm fit2109-week6
```

Demo: build and verify

Before running: predict the two output lines.

```
docker build -t fit2109-week6 .  
docker run --rm fit2109-week6
```

Expected application evidence

```
requests 2.31.0  
curl 7.x.x (...)
```

Demo: build and verify

Before running: predict the two output lines.

```
docker build -t fit2109-week6 .  
docker run --rm fit2109-week6
```

Expected application evidence

```
requests 2.31.0  
curl 7.x.x (...)
```

The same process verifies the Python package and the OS executable inside the built environment.

What it helps with

- isolated image filesystem;
- stable internal project paths;
- readable build recipe;
- one image for CI and deployment;
- publishable, replaceable artifact.

What still needs work

- base tags and repositories can change;
- images take time and storage;
- mounts, networking, and secrets need design;
- security fixes require rebuild and retest.

What it helps with

- isolated image filesystem;
- stable internal project paths;
- readable build recipe;
- one image for CI and deployment;
- publishable, replaceable artifact.

What still needs work

- base tags and repositories can change;
- images take time and storage;
- mounts, networking, and secrets need design;
- security fixes require rebuild and retest.

A container packages the application environment, not a complete operating system.

Activity 2: complete the environment

The app needs Python 3.12, requests==2.31.0, curl, and app.py. Fill the four blanks. Assume the supplied app prints the requests version and runs `curl --version`; all files are in the current directory.

requirements.txt

Dockerfile

```
FROM -----  
RUN apt-get update && -----  
WORKDIR /app  
COPY requirements.txt .  
RUN -----  
COPY app.py .  
CMD ["python", "app.py"]
```

6 minutes: explain which line handles Python, the OS package, the Python package, and source. Then write one build command and one run command that produces evidence.

Checkpoint: what Docker includes

application source + Python packages

Linux user space: runtime + OS packages + files

existing Linux kernel — not stored in the image

host or virtual hardware

- On Linux, containers use the host's Linux kernel.
- On macOS/Windows, Docker Desktop supplies a managed Linux VM; Linux containers use that VM's kernel.

When do we need a VM?

Use a VM when the requirement needs to:

- observe Ubuntu boot or systemd;
- test Windows-kernel behaviour;
- change kernel configuration or modules;
- boot and isolate a complete operating system.

application + dependencies

complete OS + its kernel

virtual hardware

host machine

VMs usually require more memory, disk, startup time, and provisioning work.

Local desktop VMs

- Oracle VirtualBox;
- VMware Workstation Pro / Fusion;
- Microsoft Hyper-V;
- Parallels Desktop.

Cloud VM services

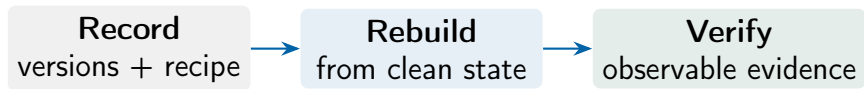
- Amazon EC2;
- Azure Virtual Machines;
- Google Compute Engine.

Start with the required operating system and CPU architecture, then check what the host and product support.

Choose by the requirement

Project requirement	Practical setup
separate Python package versions per project	requirements + venv
reuse one built Linux user space in CI boot and observe a complete OS and its kernel	Dockerfile → image VM + provisioning description

Choose the smallest practical setup that reproduces the important behaviour.



- Version control stores source, requirements, Dockerfiles, and intentional lockfiles.
- Local `.venv/` directories and containers are rebuildable installed state.
- Built images are normally published through an image registry.

Activity 3: justify the setup

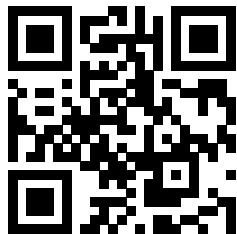
6 minutes total: 3 minutes write · 2 groups share · 1 minute compare

- A. Two Python projects need incompatible Django versions.
- B. A Python service needs Linux libraries and `curl`, and must match CI.
- C. A class must observe Ubuntu boot and `systemd` from macOS.

Give a three-line answer:

- ① **Setup:** what will you use, and which requirement does it satisfy?
- ② **Recorded description:** which file or configuration must the team keep?
- ③ **Evidence:** what can a teammate run or inspect?

For C, no VM command syntax is required; a VM configuration or provisioning description is enough.



pollev.com/fit2109

- ① Why can a later fresh install select a different package version?
- ② What does Docker add beyond a venv for this application?
- ③ Which requirement genuinely needs a VM?

What to remember

observe failure → identify difference → record setup → rebuild + verify

Do not begin with “Should I use Docker?”

Begin with the dependency or behaviour that must be reproducible.